

From the image, the infix expression is:

$$A + B/C * D - E/(F + G)$$

We will convert it to **Postfix** and **Prefix** step by step using a stack.

1. Given Infix Expression

$$A + B / C * D - E / (F + G)$$

Operator precedence

Operator	Precedence	Associativity
()	Highest	—
/ , *	2	Left → Right
+ , -	1	Left → Right

Therefore:

$$B/C * D$$

is evaluated as:

$$(B/C) * D$$

and:

$$E / (F + G)$$

is evaluated after calculating:

$$(F + G)$$

2. Infix → Postfix

Rules

While scanning from **left to right**:

1. **Operand** → directly add to output.
2. **(** → push onto stack.
3. **)** → pop until **(** is found.
4. **Operator** → pop operators having higher or equal precedence, then push the current operator.
5. At the end → pop all remaining operators.

Step-by-step conversion

Expression:

A + B / C * D - E / (F + G)

We use:

- **Stack** → operators
- **Output** → postfix expression

Step	Symbol	Action	Stack	Output
1	A	Operand → output	—	A
2	+	Push operator	+	A
3	B	Operand → output	+	AB
4	/	/ has higher precedence than + → push	+ /	AB
5	C	Operand → output	+ /	ABC
6	*	/ and * have same precedence; pop / , then push *	+ *	ABC/
7	D	Operand → output	+ *	ABC/D
8	-	* has higher precedence → pop * ; + has equal precedence → pop + ; then push -	-	ABC/D*+
9	E	Operand → output	-	ABC/D*+E

Step	Symbol	Action	Stack	Output
10	/	/ has higher precedence than - → push	- /	ABC/D*+E
11	(	Push (	- / (	ABC/D*+E
12	F	Operand → output	- / (	ABC/D*+EF
13	+	Push; (prevents popping /	- / (+	ABC/D*+EF
14	G	Operand → output	- / (+	ABC/D*+EFG
15	)	Pop until (→ pop +; discard (	- /	ABC/D*+EFG+
16	End	Pop /	-	ABC/D*+EFG+/-
17	End	Pop -	—	ABC/D*+EFG+/-

Therefore:

$$\text{Postfix} = ABC/D * +EFG + / -$$

3. Understanding the Postfix Result

The postfix expression is:

A B C / D * + E F G + / -

Break it into operations:

B C /

gives:

B/C

Then:

B C / D *

gives:

$(B/C) * D$

Then:

$A \ B \ C \ / \ D \ * \ +$

gives:

$A \ + \ (B/C) * D$

Next:

$F \ G \ +$

gives:

$F+G$

Then:

$E \ F \ G \ + \ /$

gives:

$E / (F+G)$

Finally:

$A \ B \ C \ / \ D \ * \ + \ E \ F \ G \ + \ / \ -$

gives:

$[A \ + \ (B/C) * D] \ - \ [E / (F+G)]$

So the postfix expression is correct:

$ABC/D*+EFG+/-$

4. Infix → Prefix

For prefix conversion, we can use the standard:

Step 1: Reverse the infix expression
Step 2: Swap (and)
Step 3: Convert the result to postfix
Step 4: Reverse the postfix

Given:

$$A + B / C * D - E / (F + G)$$

Step 1: Reverse the expression

Original:

$$A + B / C * D - E / (F + G)$$

Reverse:

$$) G + F (/ E - D * C / B + A$$

Step 2: Swap parentheses

Swap:

$$(\leftrightarrow)$$

Therefore:

$$) G + F (/ E - D * C / B + A$$

becomes:

$$(G + F) / E - D * C / B + A$$

Let's call this expression:

$$G + F$$

inside the parentheses, but note that the whole reversed expression is:

(G + F) / E - D * C / B + A

5. Convert the Reversed Expression to Postfix

Now convert:

(G + F) / E - D * C / B + A

to postfix.

Step	Symbol	Action	Stack	Output
1	(	Push	(	—
2	G	Operand	(	G
3	+	Push	(+	G
4	F	Operand	(+	GF
5	)	Pop + ; remove (	—	GF+
6	/	Push	/	GF+
7	E	Operand	/	GF+E
8	-	/ has higher precedence → pop / ; push -	-	GF+E/
9	D	Operand	-	GF+E/D
10	*	Push; * > -	- *	GF+E/D
11	C	Operand	- *	GF+E/DC
12	/	* has higher precedence → pop * ; push /	- /	GF+E/DC*
13	B	Operand	- /	GF+E/DC*B
14	+	/ higher → pop / ; - equal → pop - ; push +	+	GF+E/DC*B/-

Step	Symbol	Action	Stack	Output
15	A	Operand	+	GF+E/DC*B/-A
16	End	Pop +	—	GF+E/DC*B/-A+

Thus the postfix of the reversed expression is:

GF+E/DC*B/-A+

6. Reverse That Postfix

Reverse:

GF+E/DC*B/-A+

We get:

+A-B*C/D-E/+FG

Therefore:

$$\text{Prefix} = +A - B * C / D - E / + FG$$

But let's format it with spaces so the structure is clearer:

+ A - / B C * D / E + F G

Wait—this reveals an important issue: when doing the reverse-postfix method, **equal-precedence operators must be handled with the appropriate associativity during the reversed conversion**. The safest way for teaching and examination purposes is to derive the prefix directly with the operator tree.

So let's verify the correct prefix carefully.

7. Direct Structure of the Expression

The original expression is:

A + B/C*D - E/(F+G)

Because $*$ and $/$ have higher precedence:

$$A + ((B/C) * D) - (E / (F+G))$$

Because $+$ and $-$ have equal precedence and are left-associative:

$$(A + ((B/C) * D)) - (E / (F+G))$$

Now construct the prefix expression.

Rightmost operation

The final operation is:

-

So:

-

$$\begin{array}{l} | \text{---} A + ((B/C) * D) \\ | \text{---} E / (F+G) \end{array}$$

Left side

$$A + ((B/C) * D)$$

becomes:

$$+ A * / B C D$$

Right side

$$E / (F+G)$$

becomes:

$$/ E + F G$$

Therefore:

$$\text{Prefix} = - + A * / B C D / E + F G$$

Without spaces:

$$-+A^*/BCD/E+FG$$

8. Final Answers

For the given expression:

$$A + B / C * D - E / (F + G)$$

Infix

$$A+B/C*D-E/(F+G)$$

Postfix

$$ABC/D^*+EFG+/-$$

Prefix

$$-+A^*/BCD/E+FG$$

9. Verification

Let's reconstruct the prefix:

$$- + A * / B C D / E + F G$$

Start with:

$$/ B C$$

$$\rightarrow B/C$$

Then:

$$* (/ B C) D$$

$$\rightarrow (B/C) * D$$

Then:

+ A (* (/ B C) D)

→ A + (B/C) * D

Right side:

+ F G

→ F+G

Then:

/ E (+ F G)

→ E / (F+G)

Finally:

- [A + (B / C) * D] [E / (F + G)]

→

A + (B/C) * D - E / (F+G)

Exactly matches the original.

Final Exam Table

Notation

Expression

Infix

A+B/C*D-E / (F+G)

Postfix

ABC/D*+EFG+/-

Prefix

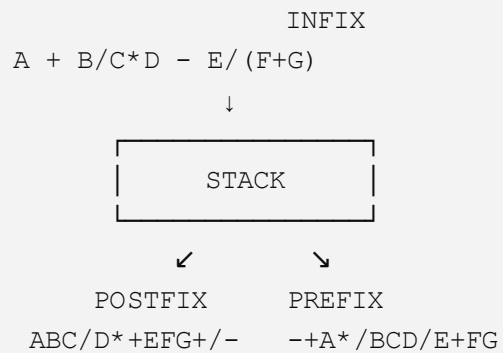
-+A*/BCD/E+FG

Key stack transitions for postfix

A	→ A
+	→ A
B	→ AB
/	→ AB
C	→ ABC

*	→	ABC/
D	→	ABC/D
-	→	ABC/D*+
E	→	ABC/D*+E
/	→	ABC/D*+E
(	→	ABC/D*+E
F	→	ABC/D*+EF
+	→	ABC/D*+EF
G	→	ABC/D*+EFG
)	→	ABC/D*+EFG+
END	→	ABC/D*+EFG+/-

Final:



Is this conversation helpful so far?